Machine Learning in Engineering Science

Week 14: Agent testing, failure cases, and oversight

Chi-Hua Jonny Yu / NCKU + LAiMM

Fall 2026 / 2026-12-07

Chapter 1

Agent testing, failure cases, and oversight

Week 14: 2026-12-07, Monday 14:00–17:00.

Test a deterministic local state machine. All messages are append-only Python list events to a fictional recipient; no mail, files, model inference or external side effect is invoked.

Learning outcome

Enforce bounded execution; test authorization and failures; bind simulated approvals to exact actions.

The meeting contains three 60-minute modules: 60 minutes explanation and 120 minutes practice. The eight core exercises and two optional challenges are in-class practice, not additional weekly graded homework.

1.1 H1: State, budgets, and trust boundaries

1.1.1 An agent is a controlled loop

An action proposal must pass the harness before changing the environment.

Worked reasoning. State; transition; attempt budget; tool permissions; tool errors; approvals; trace evidence.

Check. Where does the model lose authority?

Explanation and boundary. At every deterministic gate. A proposal cannot change the action set or enlarge its own budget.

1.1.2 Workflow and agent are not synonyms

A fixed graph is a workflow; model-directed choices add an agent decision layer.

Worked reasoning. A three-step measurement report can use a script. Do not add open-ended planning when the steps are known.

Check. Does a loop alone make a system an LLM agent?

Explanation and boundary. No. The core notebook is a deterministic harness simulator. The language-model decision component is deliberately absent.

1.1.3 Represent state before writing actions

State records attempts, observations, pending approval and terminal status.

Worked reasoning. After budget_exhausted, later proposals must not execute. A terminal status is not just a displayed message.

Check. What is the invariant after stopping?

Explanation and boundary. The executed action count and side-effect log do not change. Tests should attempt an additional action.

1.1.4 Budgets need an accounting unit

Count attempts, tool calls and declared work separately.

Worked reasoning. Three invalid proposals under a two-attempt cap must stop at the cap even though no tool ran.

Check. Can a tool-call cap guarantee short runtime?

Explanation and boundary. No. One call may hang or be expensive. Wall-clock limits and resource isolation are distinct controls.

1.1.5 Validate before dispatch

Unknown tool names must not reach an implementation or external service.

Worked reasoning. Compare a permitted mean request with a fake send_email request. The simulator sends no real message.

Check. What must the trace prove about the forbidden request?

Explanation and boundary. It was rejected before dispatch; do not rely on a downstream tool happening to fail.

1.1.6 Use typed errors with narrow recovery

Invalid input, transient tool failure and permission failure need different next steps.

Worked reasoning. A bounded retry can handle a simulated transient error; it cannot authorize a forbidden operation.

Check. Why can retries make behavior worse?

Explanation and boundary. Repeated side effects, cost and duplicate operations may occur. Retry only a declared safe operation with a bound.

1.1.7 Untrusted text cannot amend policy

Retrieved instructions are data even if they imitate a system message.

Worked reasoning. Store the fixture text and show that the allowed tools remain unchanged after it is observed.

Check. Does this test prove universal prompt-injection resistance?

Explanation and boundary. No. It tests one deterministic boundary and supplied cases; real model-mediated paths require broader adversarial evaluation.

1.1.8 Approval must name the exact action

Bind action, arguments and a unique nonce into a reviewed digest.

Worked reasoning. An approval for draft A cannot be reused for draft B with different content or destination.

Check. Why is an 'approved=true' field insufficient?

Explanation and boundary. Untrusted callers could set it themselves. Approval must come from a separate authority and be checked against the exact request.

1.1.9 Consumption prevents approval replay

A valid approval is one-use and has a declared expiration rule.

Worked reasoning. The first simulated action can succeed; the second must be rejected without a duplicate event.

Check. What limitation remains in this teaching approval?

Explanation and boundary. It models a local trusted registry, not distributed identity, cryptographic authentication, real messaging or production security.

1.1.10 Summary: make unsafe transitions impossible

Place gates in code rather than hoping a model follows instructions.

Worked reasoning. Show why a retrieved sentence cannot reach the simulated send action without a separately approved digest.

Check. What will the next hour measure?

Explanation and boundary. Expected versus actual behavior for each boundary, including errors, stops and approval misuse.

1.2 H2: Failure cases and human approval tests

1.2.1 A safety claim needs a negative test

A normal success case does not probe a forbidden transition.

Worked reasoning. Each row states initial state, proposal, expected status and permitted side effects.

Check. Why not use only example conversations?

Explanation and boundary. They may omit the actual execution boundary. Test the harness state and side-effect log directly.

1.2.2 Test status and side effects together

A rejection message is insufficient if the operation already happened.

Worked reasoning. A function can append an event and then return rejected. Checking only the string would miss the violation.

Check. Which assertion detects that bug?

Explanation and boundary. Assert that the event count and protected state are unchanged for every rejected action.

1.2.3 Task 4: probe schema edge cases

Boolean, NaN, empty values and unknown fields should fail explicitly.

Worked reasoning. A separate case reveals which check was skipped when a regression occurs.

Check. Does rejecting all requests make a good agent?

Explanation and boundary. No. Include legitimate requests to measure functionality alongside safety. Always rejecting can pass a narrow negative-only suite.

1.2.4 Task 5: exhaust the attempt cap

Use a plan longer than the cap and inspect the exact stop index.

Worked reasoning. The simulator records two consumed attempts at cap two; a later proposal returns stopped.

Check. Why test after the stop?

Explanation and boundary. It verifies terminal-state enforcement rather than merely detecting that a stop message once appeared.

1.2.5 Task 6: bind approval to arguments

Change one field after approval and verify digest mismatch.

Worked reasoning. Wrong nonce, expired approval and already-consumed approval are distinct rejection cases.

Check. Can a digest prove who approved it?

Explanation and boundary. No. It detects request mismatch relative to a trusted approval record. Identity assurance is outside this local simulator.

1.2.6 Simulated timeout is not process isolation

A fake slow-step flag exercises an error branch, not operating-system enforcement.

Worked reasoning. The notebook has no blocking network tool. A production tool requires real timeout and cancellation controls.

Check. Why avoid claiming runtime security here?

Explanation and boundary. An in-process check cannot forcibly stop arbitrary blocking code. The demonstration only tests control-flow handling.

1.2.7 Retry only a declared transient failure

A bounded retry preserves the original request and logs each attempt.

Worked reasoning. Timeout may be retryable for a read; permission_denied is never repaired by repeated calls.

Check. What if a request has a side effect?

Explanation and boundary. A retry can duplicate it. Use idempotency keys or transactional semantics, not blind re-execution.

1.2.8 Audit traces are bounded evidence

Record events without secrets, irrelevant personal data or fabricated reasoning.

Worked reasoning. Identify the first failed precondition and show that later protected events are absent.

Check. Does a trace prove the implementation was not bypassed?

Explanation and boundary. Not by itself. Tests, provenance and independent observation of the execution boundary are also needed.

1.2.9 Human review must offer a real choice

Show exact action, evidence and consequences before approval.

Worked reasoning. Name recipient, content, reason and whether the operation is reversible. Our notebook uses local fictional destinations only.

Check. Why is vague confirmation weak?

Explanation and boundary. A user cannot meaningfully authorize an unspecified or changed operation. Review scope must match execution scope.

1.2.10 Summary: safety and usefulness are separate

Report legitimate success and forbidden-action violations with denominators.

Worked reasoning. Save per-case expected/observed statuses and event counts, not just 'all passed'.

Check. What does zero observed violations mean?

Explanation and boundary. No violation was observed in this finite suite. It is not a universal guarantee of safe deployment.

1.3 H3: Oversight studio: attack the boundary

1.3.1 Probe the harness without real side effects

All destructive, messaging and approval operations remain simulated.

Worked reasoning. Restore 5 min; core tests 25 min; approval review 15 min; peer audit and export 15 min.

Check. What is not being tested?

Explanation and boundary. Real identity services, real mail APIs, operating-system sandboxes and a live language model.

1.3.2 Task 1: accept a legitimate operation

Verify the permitted mean and its independent expected value.

Worked reasoning. A dispatcher that rejects every proposal is not a useful reference even if it never performs a forbidden action.

Check. Which denominator belongs to legitimate success?

Explanation and boundary. The number of valid requests in the declared suite, not all requests including deliberate attacks.

1.3.3 Task 2: keep malicious text powerless

Pass the injection fixture as content, never as executable instructions.

Worked reasoning. The allowed-tool tuple and approval registry must not be modified by retrieved prose.

Check. What would invalidate the test?

Explanation and boundary. Changing the contract in the test to make the attack pass or executing a parsed instruction from the hostile text.

1.3.4 Task 3: prove one-use approval

Inspect the digest and event count after the first use and replay.

Worked reasoning. A replay test needs the same registry. A fresh registry would represent a different authorization context.

Check. Why does this matter for reproducibility?

Explanation and boundary. Stateful security behavior depends on event order. Preserve initial state and the exact sequence of requests.

1.3.5 Task 4: differentiate malformed inputs

Name the failing precondition for each deliberately invalid proposal.

Worked reasoning. For True, explain numeric subtype confusion; for NaN, explain nonfinite arithmetic and JSON portability.

Check. Why not catch every exception and return success?

Explanation and boundary. It would erase the failure semantics and could hide partial execution or implementation errors.

1.3.6 Task 5: inspect the terminal state

After cap exhaustion, compare the complete action log before and after.

Worked reasoning. Reset only by an explicit new session with a fresh state record and a new evidence ID.

Check. When is a reset legitimate?

Explanation and boundary. When declared as a new run, not as an invisible way to exceed the budget of an ongoing task.

1.3.7 Task 6: modify an approved request

Change one argument without issuing a new approval.

Worked reasoning. A partner should see exactly which field changed and why the original approval no longer matches.

Check. Could the caller approve its own changed request?

Explanation and boundary. Not under this contract. Approval creation is a separate trusted teaching action, not an allowed model tool.

1.3.8 Task 7: reconstruct the first failure

Follow the trace from proposal to rejection to stop.

Worked reasoning. Explain the observed predicate and result. Avoid inventing what a model internally thought.

Check. What is a defensible explanation?

Explanation and boundary. A mechanism tied to code, inputs and trace events with a stated boundary on what was observed.

1.3.9 Task 8: preserve a negative result

Export a case that fails as expected and one valid success.

Worked reasoning. The expected error must be caught by the test, not hidden by removing assertions.

Check. When does an expected rejection become a defect?

Explanation and boundary. When a valid request is rejected or an invalid request causes an unauthorized side effect.

1.3.10 Summary: extend one threat model at a time

Optional A adds a bounded retry; optional B alters one approved argument.

Worked reasoning. Name a real-world boundary that the local simulator cannot establish and propose a suitable next test.

Check. What connects to Studio I?

Explanation and boundary. The workflow will combine retrieval and numerical evidence while preserving these same permission and stop boundaries.

1.4 Invariant reference

$$0 \leq \text{consumed attempts} \leq \text{cap}.$$

A stopped session cannot execute another action. Approval must match the exact action and nonce, remain unexpired, and be unused. A digest binds bytes relative to a trusted registry; it does not establish the identity of the reviewer. Simulated timeout branches do not enforce real wall-clock cancellation.

1.5 Eight core practice tasks

1.5.1 Task 1: Accept a valid allowed calculation

Predict a valid result and verify that the permitted action is useful, not just always rejected.

Self-check. Legitimate task success is different from negative-case rejection correctness.

1.5.2 Task 2: Keep hostile source text outside authority

Record the hostile text as data and compare the action boundary before and after.

Self-check. This probes a fixed deterministic allowlist. It does not measure live LLM prompt-injection robustness.

1.5.3 Task 3: Use approval once and reject replay

The trusted reviewer binds one fictional recipient, text and nonce. Reuse the same harness for both calls.

Self-check. Exactly one list event is appended. This is a trusted local registry simulation, not cryptographic authentication.

1.5.4 Task 4: Probe data boundaries and side effects

Create a fresh harness per malformed fixture so the budget does not hide individual validation checks.

Self-check. Expected errors are caught and asserted. They are not removed to force a successful notebook run.

1.5.5 Task 5: Prove a terminal budget state

A cap of two counts invalid attempts. Probe a fourth proposal after the budget stop.

Self-check. A stopped session cannot execute a later valid proposal. Only an explicit new session can reset the budget.

1.5.6 Task 6: Reject changed or expired approvals

Compare exact-action mismatch and expiry in separate fresh harnesses; neither may append an event.

Self-check. Expiry uses an explicit simulated time input. Real clocks, identity systems and network races are outside scope.

1.5.7 Task 7: Read the first failed invariant

Construct a bounded mixed trace. Diagnose the first failure from observable predicates rather than hidden model reasoning.

Self-check. The first failed precondition is the empty numeric list. The later unapproved message never creates an event.

1.5.8 Task 8: Separate finite-suite evidence and broad claims

Record the tested invariants and explicitly name the real boundaries that were not exercised.

Self-check. Zero observed unauthorized events in a finite simulator suite is not a universal safety guarantee.

1.6 Evidence handover and summary

Enforce bounded execution; test authorization and failures; bind simulated approvals to exact actions. Save eight predictions, eight explanations and three reflections. Preserve source, data, versions, traces and one limitation. The notebook exports a draft when fields are blank; response completeness does not establish correctness. Save the edited notebook separately if changing code.

1.7 Primary references

- Building effective agents: https://www.anthropic.com/engineering/building-effective-agents.
- NIST Generative AI Profile: https://doi.org/10.6028/NIST.AI.600-1.